RouteAI Evolution Roadmap

Från ALNS-optimering till Neural Logistics Engine

Dokument: Strategisk integrationsplan
Version: 1.0
Datum: 2026-03-15

Innehållsförteckning

1. Executive Summary
2. GAP-Analys
3. Integrationspunkter
4. Prioriterad Roadmap
5. Tekniska Beslut per Fas
6. Frågeställning till Replit

Executive Summary

RouteAI har idag en solid grund med ALNS-optimering, EV-stöd och real-time WebSocket-infrastruktur. Neural Logistics Engine-visionen representerar ett paradigmskifte mot **prediktiv**, **kontinuerlig** och **personaliserad** optimering. Detta dokument kartlägger vägen från nuläge till vision genom tre faser med tydliga integrationspunkter i befintlig arkitektur.

Nyckelinsikt: Befintlig infrastruktur täcker ~40% av visionen. De största gapen finns inom prediktiv analys (GNN), kontinuerlig optimering och V2G-integration.

---

1. GAP-Analys

1.1 Nuläge vs Vision - Övergripande

Kapabilitet	RouteAI Nuläge	Neural Engine Vision	Gap-Storlek
Optimeringsalgoritm	ALNS (30s budget)	Hybrid ALNS + RL, kontinuerlig	🟡 Medium
Prediktiv analys	Ingen	GNN med väder/stadsdata	🔴 Stort
EV-hantering	10 stationer, basic SoC	V2G, regenerativ, elpris	🟡 Medium
Real-time uppdatering	WebSocket disruptions	Kontinuerlig per-sekund opt	🟡 Medium
AI-assistent	Text-chat (OpenAI)	Röststyrd, proaktiv	🟢 Litet
Personalisering	RBAC-roller	Driver Happiness, individuellt	🟡 Medium
Hållbarhet	CO_2-tracking	Buller, partiklar, V2I	🟡 Medium

1.2 Vad har vi redan? ✅

```
┌─────────────────────────────────────────────────────────┐
│                    BEFINTLIG KAPACITET                  │
├─────────────────────────────────────────────────────────┤
│ ✅ ALNS-optimering med Clarke-Wright initial solution   │
│ ✅ WebSocket-infrastruktur för real-time disruptions    │
│ ✅ PostgreSQL + Drizzle ORM (skalbar datamodell)        │
│ ✅ EV-laddstationer (10 st) med SoC-beräkning           │
│ ✅ AI chat assistant (OpenAI integration)               │
│ ✅ Sustainability dashboard med CO₂-tracking            │
│ ✅ Geofence events & route templates                    │
│ ✅ RBAC med admin/dispatcher/driver roller              │
│ ✅ React + Leaflet maps för visualisering               │
│ ✅ pnpm monorepo med TypeScript 5.9                     │
└─────────────────────────────────────────────────────────┘
```

1.3 Vad saknas? ❌

```
┌─────────────────────────────────────────────────────────────┐
│                    GAP-KOMPONENTER                          │
├─────────────────────────────────────────────────────────────┤
│ ❌ GNN-baserad prediktion (trafikmönster, väder, events)    │
│ ❌ Kontinuerlig optimering (ersätta batch 30s-körningar)    │
│ ❌ V2G-integration (vehicle-to-grid bidirektionell ström)   │
│ ❌ Elprisoptimering (spot-priser, dynamisk laddning)        │
│ ❌ Reinforcement Learning för adaptiv strategi              │
│ ❌ Driver Happiness Score (individuella preferenser)        │
│ ❌ Regenerativ bromsnings-beräkning                         │
│ ❌ V2I (Vehicle-to-Infrastructure) kommunikation            │
│ ❌ Bulleroptimering (nattliga rutter, bostadsområden)       │
│ ❌ Partikelutsläpps-tracking (PM2.5, NOx)                   │
│ ❌ Röststyrd AI-assistent                                   │
│ ❌ Monte Carlo-simuleringar (serverless)                    │
└─────────────────────────────────────────────────────────────┘
```

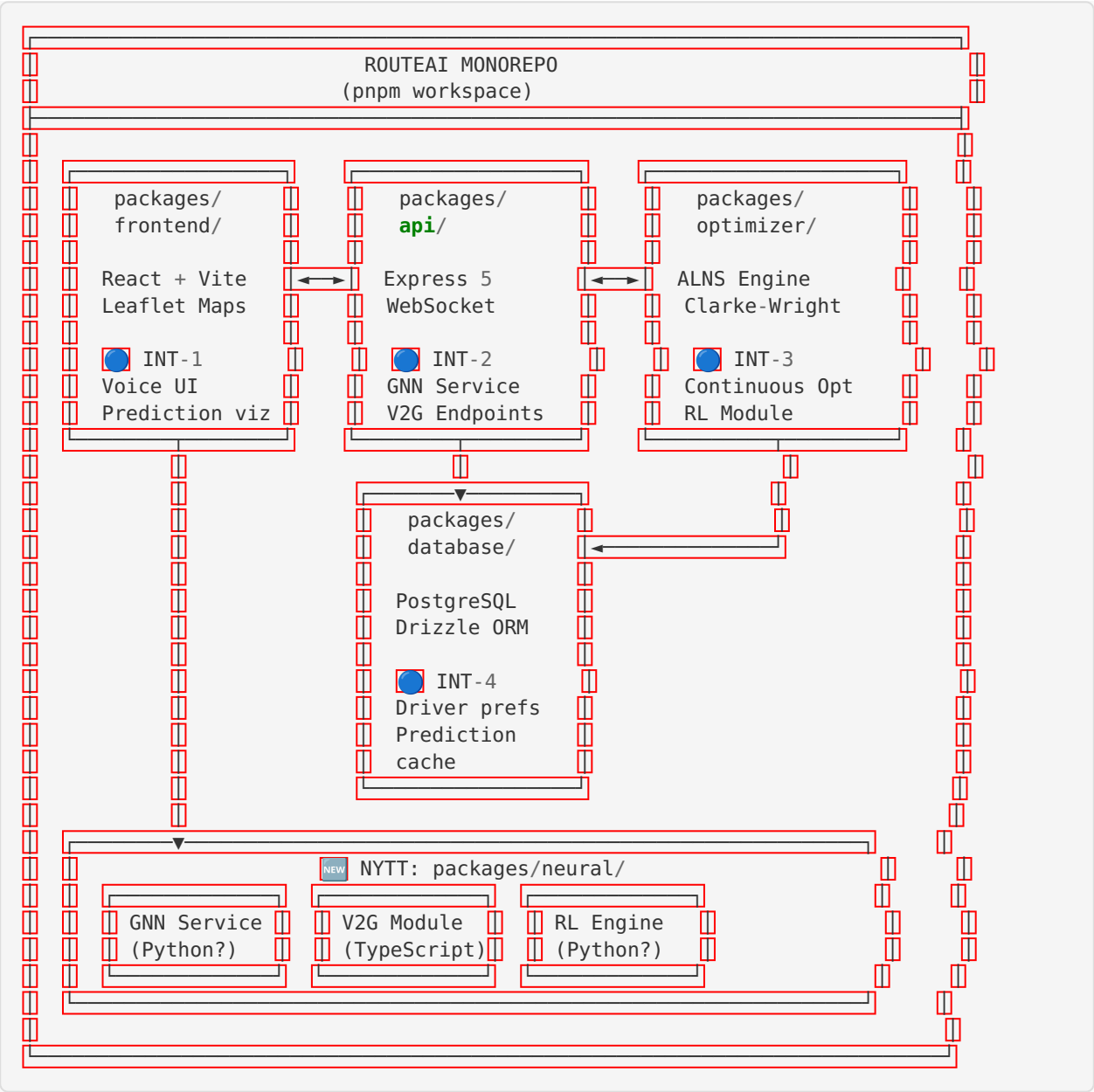
1.4 Gap-Kvantifiering

Gap-Kategori	Komplexitet	Beroenden	Affärsvärde
GNN Prediktion	Hög	Externa datakällor	⭐⭐⭐⭐⭐
Kontinuerlig Opt	Medium	Befintlig ALNS	⭐⭐⭐⭐
V2G Integration	Hög	Hardware/API:er	⭐⭐⭐⭐
Driver Personal-isering	Låg	Befintlig DB	⭐⭐⭐
RL-modul	Mycket hög	GNN + data	⭐⭐⭐⭐⭐
Röstassistent	Låg	OpenAI Whisper	⭐⭐

---

2. Integrationspunkter

2.1 Arkitekturöversikt med Integrationspunkter



2.2 Detaljerade Integrationspunkter

INT-1: Frontend Integration (packages/frontend/)

```tsx
// src/components/map/PredictionLayer.tsx (NY FIL)
// Integrationspunkt för GNN-prediktion visualisering

import { useEffect, useState } from 'react';
import { useMap } from 'react-leaflet';
import { HeatmapLayer } from 'react-leaflet-heatmap-layer-v3';

interface PredictionData {
  lat: number;
  lng: number;
  congestionScore: number;  // 0-1
  predictedDelay: number;   // minuter
  confidence: number;       // 0-1
}

export const PredictionLayer: React.FC = () => {
  const [predictions, setPredictions] = useState<PredictionData[]>([]);
  const map = useMap();

  useEffect(() => {
    // WebSocket för real-time predictions
    const ws = new WebSocket(`${WS_URL}/predictions`);
    ws.onmessage = (event) => {
      setPredictions(JSON.parse(event.data));
    };
    return () => ws.close();
  }, []);

  return (
    <HeatmapLayer
      points={predictions}
      latitudeExtractor={(p) => p.lat}
      longitudeExtractor={(p) => p.lng}
      intensityExtractor={(p) => p.congestionScore}
    />
  );
};
```

```tsx
// src/components/voice/VoiceAssistant.tsx (NY FIL)
// Integrationspunkt för röststyrd AI

import { useWhisper } from '@/hooks/useWhisper';
import { useChatContext } from '@/contexts/ChatContext';

export const VoiceAssistant: React.FC = () => {
  const { startRecording, transcript, isListening } = useWhisper();
  const { sendMessage } = useChatContext();

  useEffect(() => {
    if (transcript) {
      sendMessage(transcript, { source: 'voice' });
    }
  }, [transcript]);

  return (
    <button
      onMouseDown={startRecording}
      className={`voice-btn ${isListening ? 'recording' : ''}`}
    >
      🎤
    </button>
  );
};
```

INT-2: API Integration (packages/api/)

```ts
// src/routes/prediction.routes.ts (NY FIL)
// Integrationspunkt för GNN-prediction API

import { Router } from 'express';
import { GNNService } from '@routeai/neural';

const router = Router();
const gnnService = new GNNService();

// GET /api/predictions/traffic?bounds=lat1,lng1,lat2,lng2&horizon=30
router.get('/traffic', async (req, res) => {
  const { bounds, horizon = 30 } = req.query;
  const [lat1, lng1, lat2, lng2] = (bounds as string).split(',').map(Number);

  const predictions = await gnnService.predictTraffic({
    bounds: { lat1, lng1, lat2, lng2 },
    horizonMinutes: Number(horizon),
    includeWeather: true,
    includeEvents: true
  });

  res.json(predictions);
});

// POST /api/predictions/route-impact
router.post('/route-impact', async (req, res) => {
  const { routeId, departureTime } = req.body;
  const impact = await gnnService.predictRouteImpact(routeId, departureTime);
  res.json(impact);
});

export default router;
```

```
// src/routes/v2g.routes.ts (NY FIL)
// Integrationspunkt för V2G

import { Router } from 'express';
import { V2GService } from '@routeai/neural';

const router = Router();
const v2gService = new V2GService();

// GET /api/v2g/grid-status
router.get('/grid-status', async (req, res) => {
  const status = await v2gService.getGridStatus();
  res.json(status);
});

// POST /api/v2g/optimize-charging
router.post('/optimize-charging', async (req, res) => {
  const { vehicleId, targetSoc, deadline, allowDischarge } = req.body;

  const plan = await v2gService.optimizeCharging({
    vehicleId,
    targetSoc,
    deadline: new Date(deadline),
    allowDischarge,
    considerElectricityPrices: true
  });

  res.json(plan);
});

export default router;
```

INT-3: Optimizer Integration (packages/optimizer/)

```typescript
// src/continuous/ContinuousOptimizer.ts (NY FIL)
// Ersätter/kompletterar nuvarande 30s batch ALNS

import { ALNSEngine } from '../alns/ALNSEngine';
import { EventEmitter } from 'events';

interface OptimizationState {
  currentSolution: Solution;
  lastImprovement: Date;
  iterationCount: number;
}

export class ContinuousOptimizer extends EventEmitter {
  private state: OptimizationState;
  private alns: ALNSEngine;
  private isRunning: boolean = false;
  private microBudgetMs: number = 100; // 100ms micro-iterationer

  constructor(private readonly config: ContinuousConfig) {
    super();
    this.alns = new ALNSEngine(config.alnsConfig);
  }

  async start(initialSolution: Solution): Promise<void> {
    this.state = {
      currentSolution: initialSolution,
      lastImprovement: new Date(),
      iterationCount: 0
    };
    this.isRunning = true;

    while (this.isRunning) {
      const improved = await this.microIteration();

      if (improved) {
        this.emit('improvement', this.state.currentSolution);
      }

      // Yield för att inte blockera event loop
      await this.yieldControl();
    }
  }

  private async microIteration(): Promise<boolean> {
    const startTime = Date.now();
    let improved = false;

    while (Date.now() - startTime < this.microBudgetMs) {
      const candidate = this.alns.perturb(this.state.currentSolution);

      if (candidate.cost < this.state.currentSolution.cost) {
        this.state.currentSolution = candidate;
        this.state.lastImprovement = new Date();
        improved = true;
      }

      this.state.iterationCount++;
    }

    return improved;
  }
```

```
  injectDisruption(disruption: Disruption): void {
    // Integrera med befintlig WebSocket disruption pipeline
    this.state.currentSolution = this.alns.handleDisruption(
      this.state.currentSolution,
      disruption
    );
    this.emit('disruption-handled', this.state.currentSolution);
  }

  stop(): Solution {
    this.isRunning = false;
    return this.state.currentSolution;
  }

  private yieldControl(): Promise<void> {
    return new Promise(resolve => setImmediate(resolve));
  }
}
```

```
// src/rl/RLModule.ts (NY FIL)
// Reinforcement Learning integration

import { spawn } from 'child_process';

export class RLModule {
  private pythonProcess: ChildProcess | null = null;

  async initialize(): Promise<void> {
    // Starta Python RL-server som sidecar
    this.pythonProcess = spawn('python', [
      '-m', 'routeai_rl.server',
      '--port', '50051'
    ]);
  }

  async selectOperator(state: ALNSState): Promise<OperatorSelection> {
    // gRPC-anrop till Python RL-modul
    const response = await this.grpcClient.selectOperator({
      destroyHistory: state.destroyHistory,
      repairHistory: state.repairHistory,
      currentCost: state.currentCost,
      temperature: state.temperature
    });

    return {
      destroyOperator: response.destroyOp,
      repairOperator: response.repairOp,
      confidence: response.confidence
    };
  }
}
```

INT-4: Database Integration (packages/database/)

```
// src/schema/driver-preferences.ts (NY FIL)
// Driver Happiness personalisering

import { pgTable, serial, integer, real, jsonb, timestamp } from 'drizzle-orm/pg-
core';
import { drivers } from './drivers';

export const driverPreferences = pgTable('driver_preferences', {
  id: serial('id').primaryKey(),
  driverId: integer('driver_id').references(() => drivers.id),

  // Tidspreferenser
  preferredStartTime: timestamp('preferred_start_time'),
  preferredEndTime: timestamp('preferred_end_time'),
  avoidNightDriving: boolean('avoid_night_driving').default(false),

  // Ruttpreferenser
  maxDrivingHours: real('max_driving_hours').default(8),
  preferHighways: boolean('prefer_highways').default(true),
  avoidUrbanAreas: boolean('avoid_urban_areas').default(false),

  // Pauspreferenser
  breakFrequencyMinutes: integer('break_frequency_minutes').default(120),
  preferredBreakDuration: integer('preferred_break_duration').default(15),

  // Lärda preferenser (från beteendeanalys)
  learnedPreferences: jsonb('learned_preferences').$type<{
    avgSpeed: number;
    preferredRouteTypes: string[];
    breakPatterns: { hour: number; probability: number }[];
    stressIndicators: { trafficThreshold: number; timeThreshold: number };
  }>(),

  // Driver Happiness Score (beräknad)
  happinessScore: real('happiness_score').default(0.5),

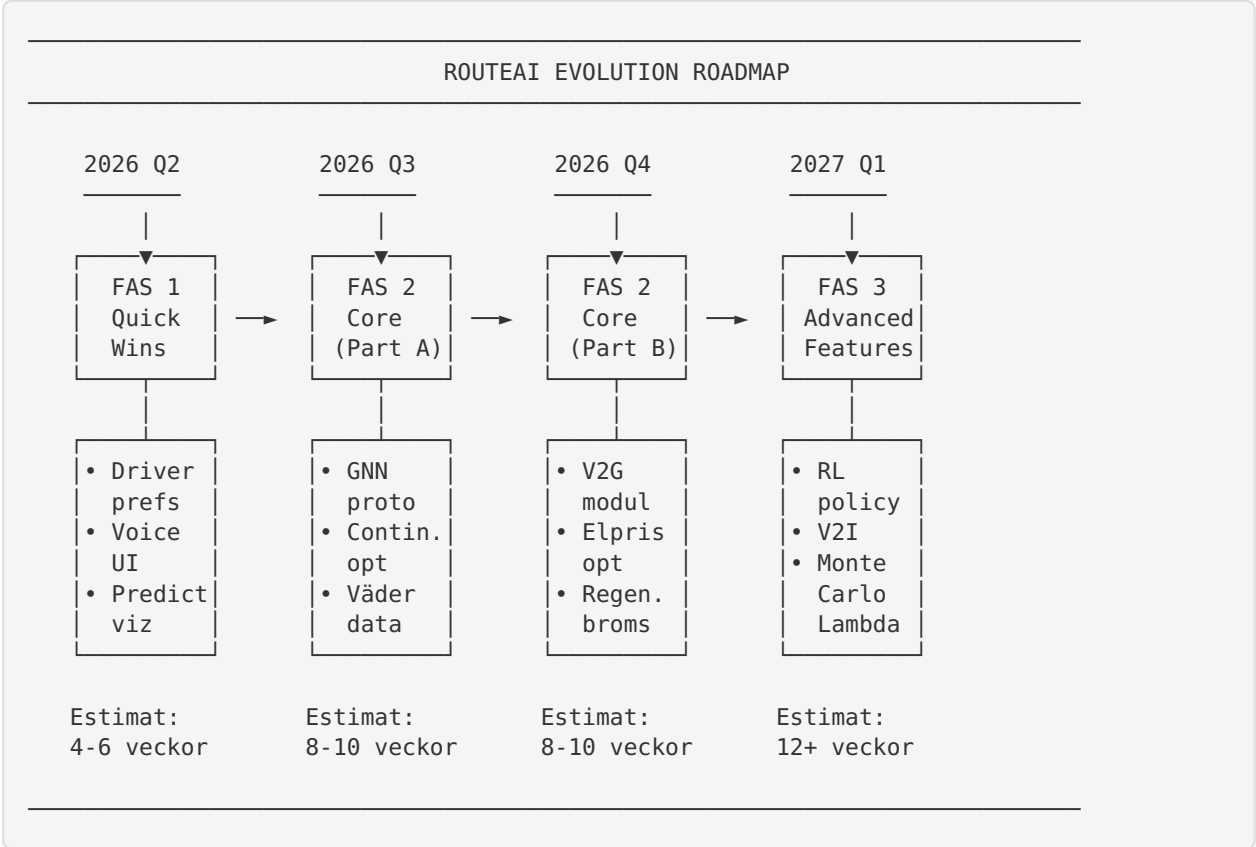
  updatedAt: timestamp('updated_at').defaultNow()
});

export const predictionCache = pgTable('prediction_cache', {
  id: serial('id').primaryKey(),
  gridCell: text('grid_cell').notNull(), // geohash
  predictionType: text('prediction_type').notNull(), // 'traffic', 'weather', 'events'
  predictions: jsonb('predictions'),
  validFrom: timestamp('valid_from').notNull(),
  validTo: timestamp('valid_to').notNull(),
  createdAt: timestamp('created_at').defaultNow()
});

export const v2gSessions = pgTable('v2g_sessions', {
  id: serial('id').primaryKey(),
  vehicleId: integer('vehicle_id').references(() => vehicles.id),
  stationId: integer('station_id'),
  sessionType: text('session_type').notNull(), // 'charge', 'discharge', 'idle'
  startTime: timestamp('start_time').notNull(),
  endTime: timestamp('end_time'),
  energyKwh: real('energy_kwh'),
  revenueEur: real('revenue_eur'),
  gridContribution: jsonb('grid_contribution')
});
```

3. Prioriterad Roadmap

3.1 Visuell Roadmap



3.2 Fas 1: Quick Wins (4-6 veckor)

Mål: Lägg grund för framtida features med minimal risk till befintligt system.

#	Feature	Beskrivning	Effort	Värde
1.1	Driver Preferences	Utöka driver-schema med preferenser	3d	⭐⭐⭐
1.2	Voice UI (beta)	OpenAI Whisper integration i chat	5d	⭐⭐
1.3	Prediction Visualization	Leaflet heatmap-layer (mock data först)	3d	⭐⭐⭐
1.4	Continuous Opt Foundation	Refaktorera ALNS för micro-iterationer	5d	⭐⭐⭐⭐
1.5	Neural Package Setup	Skapa `packages/neural/` med struktur	2d	⭐⭐⭐⭐⭐
1.6	Elpris API-integration	Nordpool spotpris polling	3d	⭐⭐⭐

Leverabler:
- Databas-migration för driver_preferences
- VoiceAssistant React-komponent
- PredictionLayer med mock-data toggle
- ContinuousOptimizer MVP
- packages/neural/ monorepo-struktur
- Nordpool API-integration

3.3 Fas 2: Core Enhancements (16-20 veckor)

Fas 2A: GNN & Continuous Optimization (8-10 veckor)

#	Feature	Beskrivning	Effort	Värde
2.1	GNN Prototype	Python service med PyTorch Geometric	3w	⭐⭐⭐⭐⭐
2.2	Weather Integration	SMHI/OpenWeatherMap → GNN input	1w	⭐⭐⭐⭐
2.3	Event Data Pipeline	Google Events / Ticketmaster API	1w	⭐⭐⭐
2.4	Continuous Opt Production	Full integration med WebSocket	2w	⭐⭐⭐⭐
2.5	Prediction API	REST + WebSocket endpoints	1w	⭐⭐⭐⭐
2.6	Driver Behavior Learning	ML för preferens-inlärning	2w	⭐⭐⭐

Fas 2B: V2G & Energioptimering (8-10 veckor)

#	Feature	Beskrivning	Effort	Värde
2.7	V2G Data Model	v2g_sessions, grid_status tabeller	1w	⭐⭐⭐⭐
2.8	Charging Optimizer	Elprisbaserad laddningsplanering	3w	⭐⭐⭐⭐⭐
2.9	Regenerative Estimation	Beräkna bromsenergi per rutt	2w	⭐⭐⭐
2.10	V2G Simulator	Mock-grid för testning	1w	⭐⭐⭐
2.11	Grid API Integration	Anslut till faktisk grid-operator	2w	⭐⭐⭐⭐

3.4 Fas 3: Advanced Features (12+ veckor)

#	Feature	Beskrivning	Effort	Värde
3.1	RL Policy Training	Träna adaptiv ALNS-strategi	4w	⭐⭐⭐⭐⭐
3.2	V2I Integration	Trafikljus-kommunikation	4w	⭐⭐⭐
3.3	Monte Carlo Lambda	AWS serverless simuleringar	2w	⭐⭐⭐⭐
3.4	Noise Optimization	Bullerkartor, nattoptimering	2w	⭐⭐
3.5	Particle Emissions	PM2.5/NOx tracking	2w	⭐⭐
3.6	Full Voice Assistant	Proaktiva meddelanden, naturlig dialog	2w	⭐⭐⭐

---

4. Tekniska Beslut per Fas

4.1 Fas 1: Quick Wins

NPM-paket

```
// package.json additions
{
  "dependencies": {
    // Frontend
    "react-leaflet-heatmap-layer-v3": "^3.0.0",
    "openai": "^4.x",  // redan finns, använd för Whisper

    // Backend
    "node-fetch": "^3.x",  // Nordpool API
    "ws": "^8.x"  // redan finns
  }
}
```

API-endpoints

Metod	Endpoint	Fas 1 Implementation
GET	`/api/drivers/:id/preferences`	Ny
PUT	`/api/drivers/:id/preferences`	Ny
POST	`/api/voice/transcribe`	Ny (Whisper proxy)
GET	`/api/energy/prices`	Ny (Nordpool cache)
WS	`/ws/predictions`	Ny (mock data)

Databas-migrations

```
-- Migration: 001_driver_preferences.sql
CREATE TABLE driver_preferences (
  id SERIAL PRIMARY KEY,
  driver_id INTEGER REFERENCES drivers(id),
  preferred_start_time TIME,
  preferred_end_time TIME,
  avoid_night_driving BOOLEAN DEFAULT FALSE,
  max_driving_hours REAL DEFAULT 8,
  prefer_highways BOOLEAN DEFAULT TRUE,
  break_frequency_minutes INTEGER DEFAULT 120,
  happiness_score REAL DEFAULT 0.5,
  learned_preferences JSONB,
  updated_at TIMESTAMP DEFAULT NOW()
);

-- Migration: 002_energy_prices_cache.sql
CREATE TABLE energy_prices (
  id SERIAL PRIMARY KEY,
  price_area TEXT NOT NULL,  -- SE1, SE2, SE3, SE4
  timestamp TIMESTAMP NOT NULL,
  price_eur_mwh REAL NOT NULL,
  created_at TIMESTAMP DEFAULT NOW(),
  UNIQUE(price_area, timestamp)
);
```

Monorepo-struktur

```
packages/
├── api/
├── database/
├── frontend/
├── optimizer/
└── neural/          ← NY
    ├── package.json
    ├── tsconfig.json
    ├── src/
    │   ├── index.ts
    │   ├── types.ts
    │   ├── gnn/           (placeholder)
    │   ├── v2g/           (placeholder)
    │   └── rl/            (placeholder)
    └── python/            ← För framtida ML
        └── requirements.txt
```

4.2 Fas 2: Core Enhancements

NPM-paket

```
{
  "dependencies": {
    // packages/neural
    "@grpc/grpc-js": "^1.9.0",
    "@grpc/proto-loader": "^0.7.0",
    "onnxruntime-node": "^1.16.0",  // Kör GNN i Node

    // packages/api
    "axios": "^1.x"  // Weather/Events APIs
  }
}
```

Python Dependencies (packages/neural/python/)

```
# requirements.txt
torch>=2.0.0
torch-geometric>=2.4.0
pytorch-lightning>=2.0.0
grpcio>=1.59.0
grpcio-tools>=1.59.0
onnx>=1.15.0
numpy>=1.24.0
pandas>=2.0.0
scikit-learn>=1.3.0
```

API-endpoints

Metod	Endpoint	Beskrivning
GET	`/api/predictions/traffic`	GNN traffic prediction
POST	`/api/predictions/route-impact`	Predict delay för rutt
GET	`/api/weather/forecast`	Aggregerad väderdata
GET	`/api/events/upcoming`	Events som påverkar trafik
GET	`/api/v2g/grid-status`	Aktuell grid-status
POST	`/api/v2g/optimize-charging`	Laddningsplan
GET	`/api/v2g/sessions/:vehicleId`	V2G-historik

Databas-migrations

```sql
-- Migration: 003_prediction_cache.sql
CREATE TABLE prediction_cache (
  id SERIAL PRIMARY KEY,
  grid_cell TEXT NOT NULL,
  prediction_type TEXT NOT NULL,
  predictions JSONB NOT NULL,
  valid_from TIMESTAMP NOT NULL,
  valid_to TIMESTAMP NOT NULL,
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_prediction_lookup
ON prediction_cache(grid_cell, prediction_type, valid_from, valid_to);

-- Migration: 004_v2g_sessions.sql
CREATE TABLE v2g_sessions (
  id SERIAL PRIMARY KEY,
  vehicle_id INTEGER REFERENCES vehicles(id),
  station_id INTEGER,
  session_type TEXT NOT NULL CHECK (session_type IN ('charge', 'discharge', 'idle')),
  start_time TIMESTAMP NOT NULL,
  end_time TIMESTAMP,
  energy_kwh REAL,
  revenue_eur REAL,
  grid_contribution JSONB,
  created_at TIMESTAMP DEFAULT NOW()
);

-- Migration: 005_learned_behaviors.sql
CREATE TABLE driver_behavior_logs (
  id SERIAL PRIMARY KEY,
  driver_id INTEGER REFERENCES drivers(id),
  event_type TEXT NOT NULL,
  event_data JSONB NOT NULL,
  timestamp TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_behavior_driver ON driver_behavior_logs(driver_id, timestamp);
```

4.3 Fas 3: Advanced Features

NPM-paket

```
{
  "dependencies": {
    // Serverless
    "@aws-sdk/client-lambda": "^3.x",
    "@aws-sdk/client-s3": "^3.x",

    // V2I simulation
    "mqtt": "^5.x"  // MQTT för V2I-kommunikation
  }
}
```

Serverless (AWS Lambda)

```yaml
# serverless.yml (ny fil i root)
service: routeai-monte-carlo

provider:
  name: aws
  runtime: python3.11
  region: eu-north-1
  memorySize: 1024
  timeout: 300

functions:
  monteCarloSimulation:
    handler: lambda/monte_carlo.handler
    events:
      - http:
          path: /simulate
          method: post
    environment:
      SCENARIO_BUCKET: ${self:custom.bucketName}

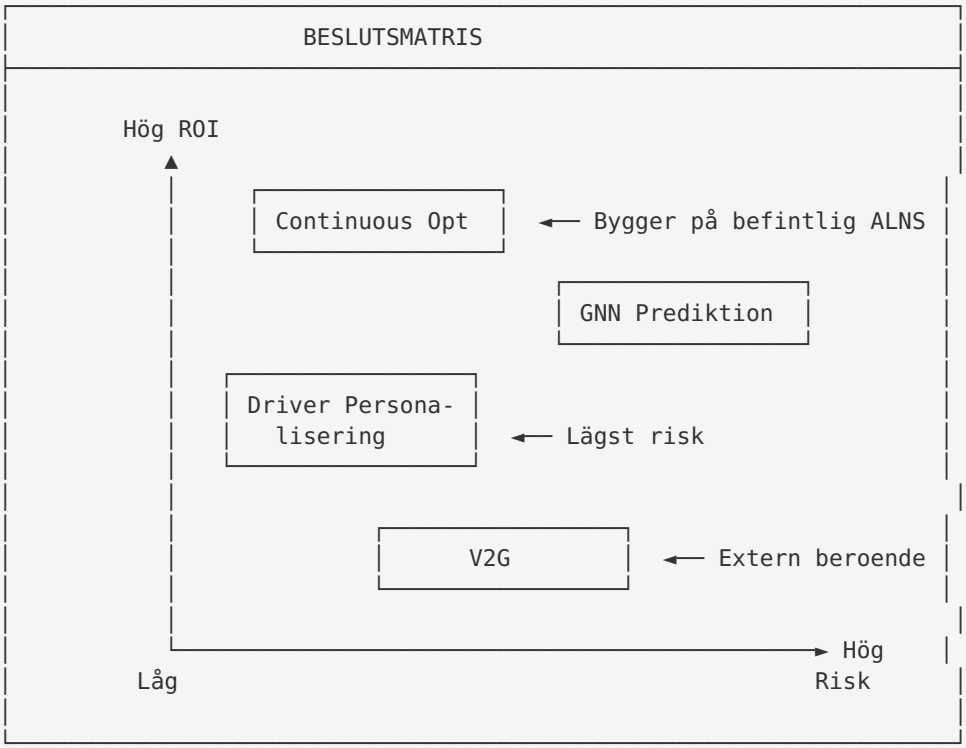
custom:
  bucketName: routeai-simulations-${self:provider.stage}
```

---

5. Frågeställning till Replit

5.1 Sammanfattning av Beslutspunkter

Vi står inför ett arkitekturellt vägskäl där vi måste balansera:



5.2 Konkret Frågeställning

📋 FRÅGESTÄLLNING TILL REPLIT

Kontext:
Vi har ett fungerande RouteAI-system med ALNS-optimering, EV-stöd, och WebSocket real-time pipeline. Vi vill evolvera mot Neural Logistics Engine-visionen med GNN-prediktion, kontinuerlig optimering, V2G-integration och driver personalisering.

Huvudfråga:

> **Vilket av följande fyra utvecklingsspår bör vi prioritera som första större investering efter Quick Wins-fasen, givet att vi vill maximera kundvärde och minimera teknisk risk?**

Alternativ A: GNN-baserad Prediktion

- **För:** Differentiator, framtidssäker, möjliggör proaktiv routing
- **Mot:** Kräver ML-kompetens, datakällor, Python-sidecar
- **Risk:** Medium-Hög (ny teknologi)
- **ROI:** Hög (om det fungerar)

Alternativ B: Kontinuerlig Optimering

- **För:** Bygger på befintlig ALNS, direkt förbättring av core product
- **Mot:** Kräver arkitekturförändringar, CPU-intensivt
- **Risk:** Låg-Medium (evolution, inte revolution)
- **ROI:** Hög (mätbar förbättring)

Alternativ C: V2G-integration

- **För:** Unik marknadsfördel, hållbarhetsfokus, intäktsmöjlighet
- **Mot:** Beroende av externa partners/hardware, lång säljcykel
- **Risk:** Hög (externa beroenden)
- **ROI:** Potentiellt mycket hög (ny intäktsström)

Alternativ D: Driver Personalisering

- **För:** Lägst risk, byggbar med befintlig stack, direkt drivarnöjdhet
- **Mot:** Mindre “wow-faktor”, inkrementell förbättring
- **Risk:** Låg
- **ROI:** Medium (kundretention)

Delfrågor:

1. **Teknisk arkitektur:** Bör vi introducera Python (för GNN/RL) som sidecar-service nu, eller hålla oss till TypeScript så länge det går?
2. **Risk vs Innovation:** Är vår nuvarande användarbas redo för prediktiv routing, eller bör vi först perfekta grundfunktionaliteten (kontinuerlig opt + personalisering)?
3. **Resursallokering:** Med begränsad utvecklingskapacitet, ska vi gå djupt på ett spår eller bredare med parallella workstreams?
4. **MVP-definition:** Vad är absolut minimum för att validera varje spår med riktiga användare?

Vår Rekommendation (för diskussion):

Vi lutar åt **Alternativ B (Kontinuerlig Optimering)** som primärt fokus, med **Alternativ D (Driver Personalisering)** som sekundärt parallellt spår. Detta ger:

- ✅ Direkt förbättring av core product
- ✅ Låg-medium risk
- ✅ Byggbart med befintlig TypeScript-stack
- ✅ Skapar grund för framtida GNN-integration
- ✅ Driver Happiness ger snabb feedback-loop

GNN/V2G skjuts till Fas 2B/3 när grundplattformen är stabilare.

Inväntar er input för att fatta beslut och starta implementation.

Dokument framtaget 2026-03-15
Nästa steg: Replits respons → Sprint Planning → Implementation